

SLIIT
SE3082 – Parallel Computing



Lab08
Message Passing Interface (MPI)

Akash D
IT24101853

Exercise 2: Parallelizing Summation (1 to 10,000,000)

Implementation Details:

The problem size (N = 10,000,000) was divided equally among the available processors. Each processor was assigned a specific starting and ending range based on its rank.

- Processor 0 calculates a local sum for its chunk.
- Processor 1 calculates a local sum for its chunk.
- MPI_Reduce was utilized to gather all local sums and aggregate them into a global_sum at Processor 0 using the MPI_SUM operation.

EXECUTION RESULTS (4 PROCESSORS):

TOTAL SUM: 50000005000000

TIME TAKEN: 0.002415 SECONDS WITH 4 PROCESSOR

Exercise 3: monte carlo pi calculation

The monte carlo method involves generating random coordinates (x, y) and checking if they fall within a unit circle.

- The total 10,000,000 iterations were divided across processors.
- To prevent duplicate sequences, each processor seeded its random number generator uniquely using its rank: srand(time(null) + rank * 1000).
- Each processor maintained a local count of points inside the circle.
- Mpi_reduce aggregated the total count to processor 0, which then computed π .

EXECUTION RESULTS (4 PROCESSORS):

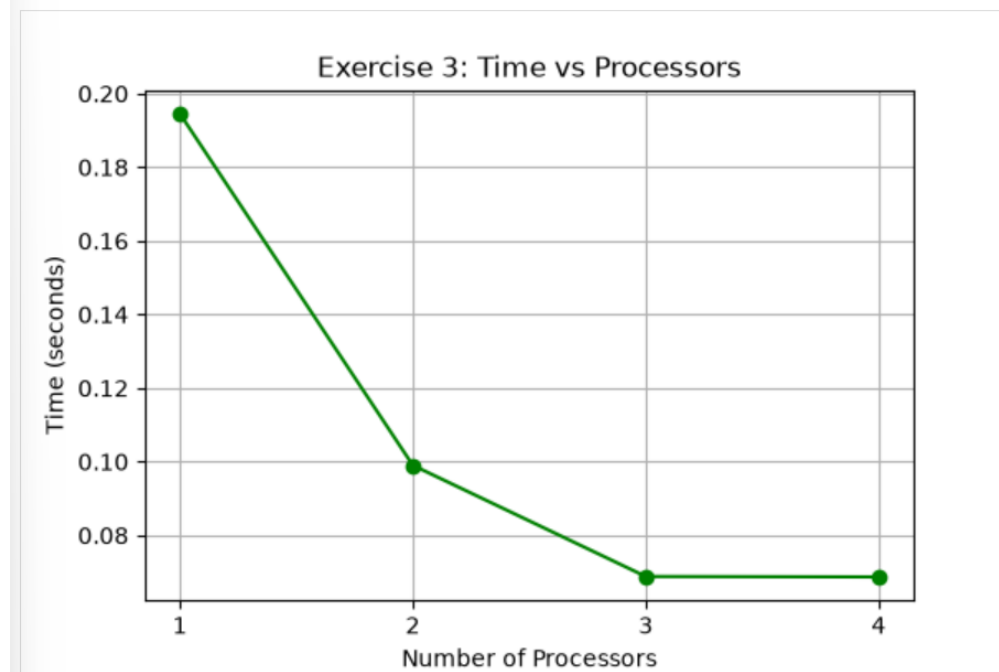
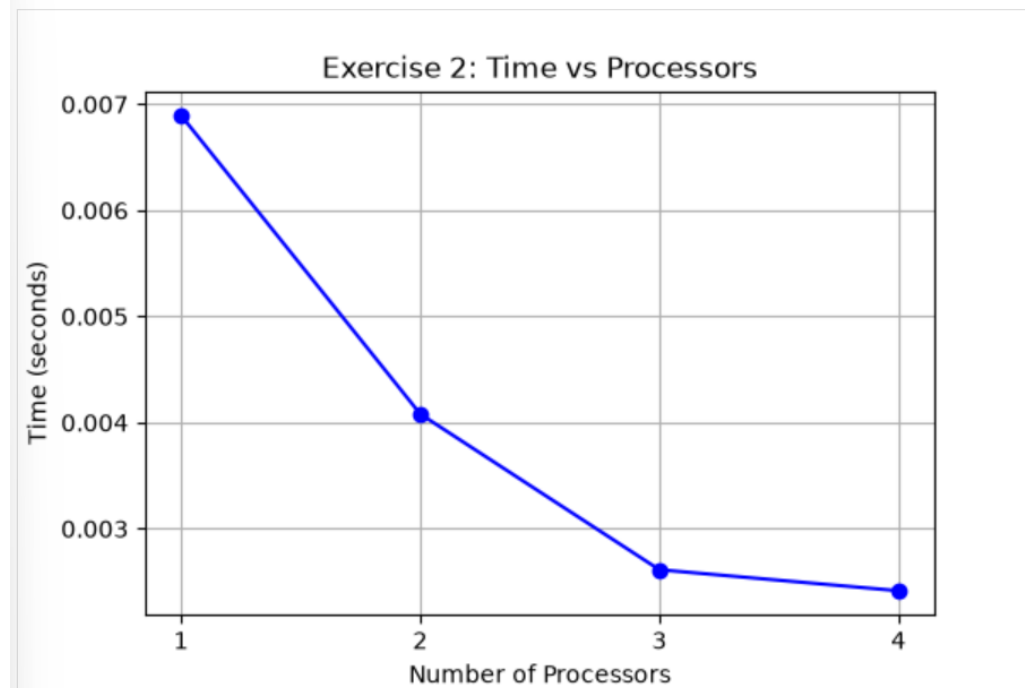
calculated pi: 3.140735

Time taken: 0.068682 seconds with 4 processors

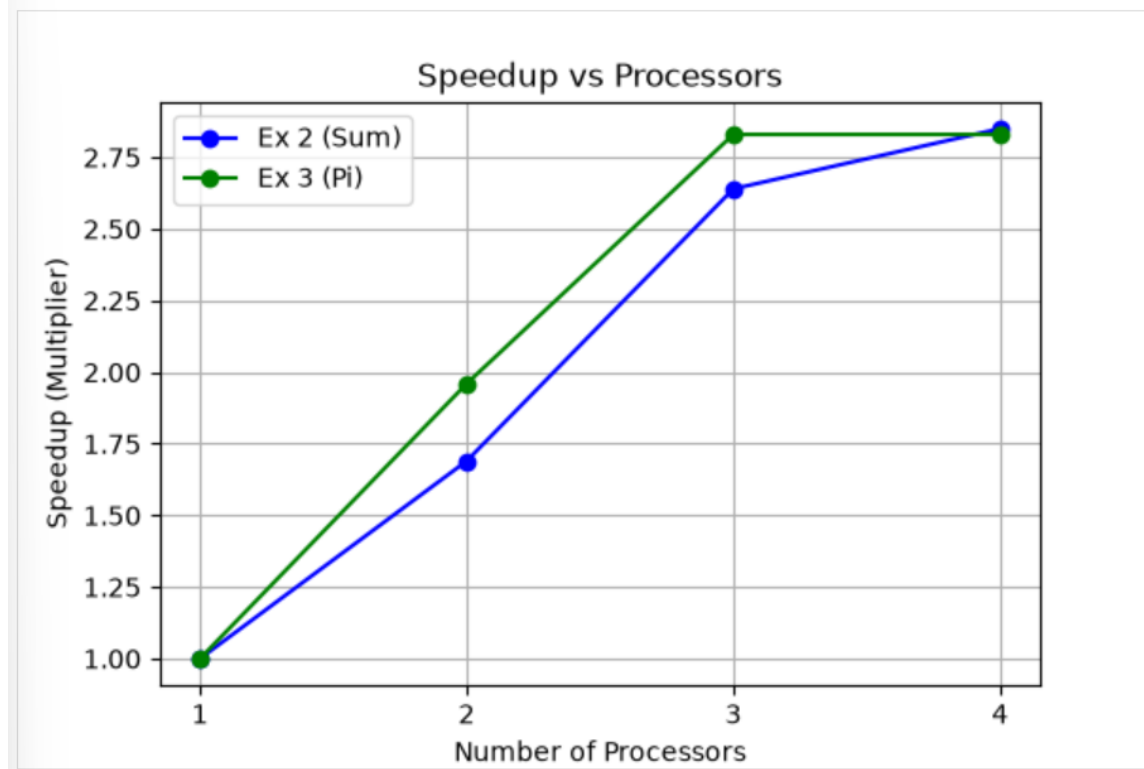
Exercise 4: performance analysis (time & speedup)

Both programs were executed using 1, 2, 3, and 4 processors to benchmark their execution times. The speedup was calculated using the formula $s = t_1 / t_p$.

Execution Time vs Processors



Speedup Graph



Exercise 5: Message Passing Modifications

Part 1: Mismatched Source/Destination

Observation: Rank 1 was configured to send a message to Rank 2 using MPI_Ssend (Synchronous Send). However, Rank 3 was configured to wait for a message from Rank 1. Because MPI_Ssend requires the destination to actively receive the message before continuing, Rank 1 blocked indefinitely. Concurrently, Rank 3 blocked indefinitely waiting for Rank 1. This resulted in a **deadlock**.

Part 2: Buffered Send (BSend)

Implementation: MPI_Buffer_attach was used to allocate a specific memory buffer. The send operation was changed from MPI_Ssend to MPI_Bsend. This allowed the sending processor to package its array, place it in the buffer, and immediately resume execution without waiting for the receiving processor to call MPI_Recv.

Exercise 6: Implementing MPI_ANY_SOURCE

Implementation Details:

Instead of an automated reduction, all worker processors computed their local Pi counts and transmitted them via MPI_Send to Processor 0. Processor 0 utilized a for-loop calling MPI_Recv with the MPI_ANY_SOURCE parameter.

This allowed Processor 0 to dynamically receive counts in the exact order that the worker processors finished their execution, rather than being forced to wait for them sequentially.

Exercise 7: MPI_ANY_SOURCE with Buffered Sends

Implementation Details:

The MPI_Send commands on the worker processors were replaced with MPI_Bsend. Each worker allocated its own local buffer, executed its Monte Carlo calculation, dispatched its count into the buffer, and exited immediately. Processor 0 continued to receive these buffered messages dynamically utilizing MPI_ANY_SOURCE.